

MINISTRY OF EDUCATION OF REPUBLIC OF MOLDOVA
TECHNICAL UNIVERSITY OF MOLDOVA
FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
SOFTWARE ENGINEERING DEPARTMENT

EMBEDDED SYSTEMS
LABORATORY WORK #10 - PART 2

Automatic Temperature Control with Assisted Fan Cooling

Author:

Sava LUCHIAN
std. gr. FAF-233

Verified:

Alexei MARTINIUC

Chisinau 2026

Technical Task

Purpose of the Laboratory

The purpose of this laboratory is to extend the temperature-control application from Part 1 with an active cooling stage. The base system remains an **Arduino Uno** application with a **10k NTC thermistor**, potentiometer setpoint, heater resistor, LCD, and `printf()` telemetry. Part 2 adds a **DC motor fan driven through L293D**; the fan speed increases gradually when the controlled resistor assembly becomes too hot and decreases gradually when the temperature returns to a safe region.

Objectives

1. Keep the modular ON-OFF heater controller from Part 1.
2. Add an L293D-based fan actuator controlled with PWM.
3. Implement gradual fan speed increase when the measured temperature exceeds the cooling threshold.
4. Keep sequential periodic tasks rather than FreeRTOS.
5. Display and report the final runtime values: setpoint, temperature, heater state, and fan speed.

Implemented Variant

The implemented Part 2 system is based on the final Lab 9 firmware and hardware:

- **Sensor input:** thermistor divider on A0.
- **Setpoint input:** potentiometer on A1 mapped to 20.0–80.0 °C.
- **Heating controller:** ON-OFF with hysteresis half-band of 1.0 °C.
- **Heating actuator:** resistor switched from D8.
- **Cooling actuator:** fan motor controlled by L293D with PWM on D11.
- **Reporting:** LCD + `printf()` CSV in the format `setpoint,temperature,heater,fan`.

1 Domain Analysis

1.1 Technological Context and Application Domain

The thermal process in this laboratory consists of a resistive heater and nearby thermistor. Part 1 controlled the heater with ON-OFF hysteresis. Part 2 adds active

cooling, which improves the response when the temperature exceeds the safe or desired range. This is a common embedded-control pattern: a simple binary heating loop is combined with proportional-like cooling effort.

The L293D driver is used because the Arduino pin cannot directly drive a DC motor safely. The Uno provides direction and PWM commands, while the motor driver handles the fan current. This also keeps the motor electrically separated from the microcontroller output pins.

1.2 Hardware Components and Justification

Component		Qty	Role in the application
Arduino (ATmega328P)	Uno	1	Main MCU for acquisition, control, actuation, LCD handling, and serial reporting.
NTC thermistor 10k		1	Analog temperature sensing element.
Fixed resistor 10k		1	Forms the thermistor voltage divider on A0.
Potentiometer 10k		1	User-adjustable setpoint source on A1.
Heater resistor 220 Ω		1	Resistive thermal source controlled from D8.
L293D motor driver		1	Drives the cooling fan from Uno digital pins.
DC motor with fan		1	Active cooling output with gradual speed increase.
LCD1602 + support resistors		1	Local status display for setpoint, temperature, heater, and fan state.
USB serial terminal / plotter		1	Receives <code>printf()</code> telemetry.

Table 1: Hardware components used in Lab 10 Part 2

1.3 Software Components and Technologies

- **PlatformIO + VS Code:** project build and dependency management.
- **Arduino framework:** GPIO, ADC, timing, PWM, serial, LCD support.
- **AVR-libc STDIO:** stdout redirected to serial through `fdev_setup_stream`.
- **LiquidCrystal:** 1602 LCD driver in 4-bit mode.
- **Wokwi:** simulation environment for the Uno wiring and runtime checks.

2 Conceptualisation and Design

2.1 Technical Requirements

ID	Category	Requirement	Type
R-F-01	Sensor	System shall read temperature from a thermistor divider.	Func.
R-F-02	Setpoint	System shall read setpoint from potentiometer on A1.	Func.
R-F-03	Heating	System shall implement ON-OFF hysteresis for the heater.	Func.
R-F-04	Cooling	System shall drive a fan through L293D when temperature is too high.	Func.
R-F-05	Cooling	Fan speed shall increase gradually as temperature rises above the cooling threshold.	Func.
R-F-06	Display	LCD shall show setpoint, temperature, heater state, and fan level.	Func.
R-F-07	Reporting	Runtime values shall be sent through <code>printf()</code> as CSV.	Func.
R-NF-01	Timing	The application shall use sequential periodic tasks.	Non-func.
R-NF-02	Modularity	Code shall be split into reusable modules.	Non-func.
R-NF-03	Platform	Firmware shall fit Arduino Uno resource limits.	Non-func.

Table 2: Technical requirements for Lab 10 Part 2

2.2 Control Design

The heater is controlled by hysteresis:

$$Heater = \begin{cases} ON, & T \leq SP - H \\ OFF, & T \geq SP + H \\ previous, & SP - H < T < SP + H \end{cases}$$

The fan is controlled by a gradual cooling law. It remains off below the cooling start threshold, then increases toward full speed as the measured temperature rises:

$$Fan_{target} = \begin{cases} 0\%, & T \leq SP + 1.0 \\ 100 \cdot \frac{T - (SP + 1.0)}{9.0}, & SP + 1.0 < T < SP + 10.0 \\ 100\%, & T \geq SP + 10.0 \end{cases}$$

The actual fan output does not jump immediately to the target. It changes by 5% every 100 ms, so the fan accelerates and decelerates gradually.

2.3 Timing Design

Activity	Period	Trigger	Handler
Setpoint update	100 ms	<code>millis()</code> elapsed	<code>runSetpointTask()</code>
Acquisition update	100 ms	<code>millis()</code> elapsed	<code>runAcquisitionTask()</code>
Control update	250 ms	<code>millis()</code> elapsed	<code>runControlTask()</code>
Report update	500 ms	<code>millis()</code> elapsed	<code>runReportTask()</code>
Fan ramp update	100 ms	<code>millis()</code> elapsed	<code>fan actuator tick()</code>

Table 3: Sequential periodic timing model for Lab 10 Part 2

3 Hardware and Software Implementation

3.1 Electrical Schematic and Hardware Setup

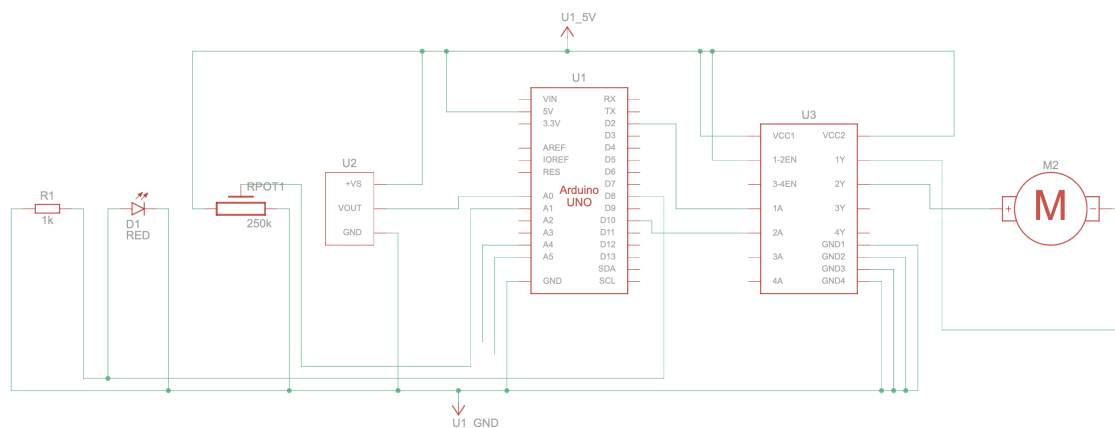


Figure 1: Electrical circuit schematic for Lab 10 Part 2

Pin assignment:

- A0 – thermistor divider node
- A1 – potentiometer signal
- D8 – heater resistor output
- D2 – L293D IN1
- D10 – L293D IN2
- D11 – L293D EN1,2 (PWM)

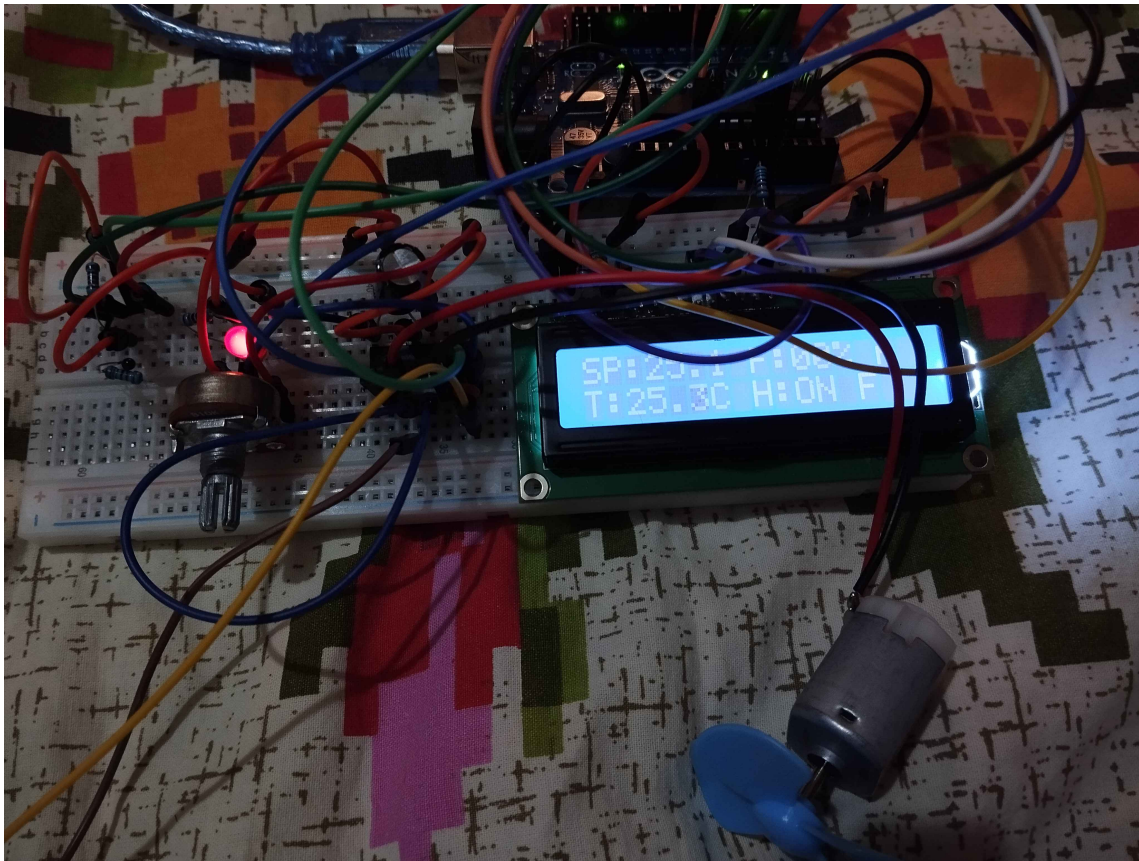


Figure 2: Hardware setup while the heater resistor is active

- D7 – LCD RS
- D6 – LCD E
- D5 – LCD D4
- D4 – LCD D5
- D3 – LCD D6
- D13 – LCD D7
- D0/D1 – USB serial UART for STDIO output

3.2 Project Directory Structure

```
lab10/  
|-- report/  
|   |-- main.tex  
|   |-- electrical_schema_10.png  
|   |-- hardware1.jpg  
|   |-- hardware2.jpg  
|   '-- terminal.png
```

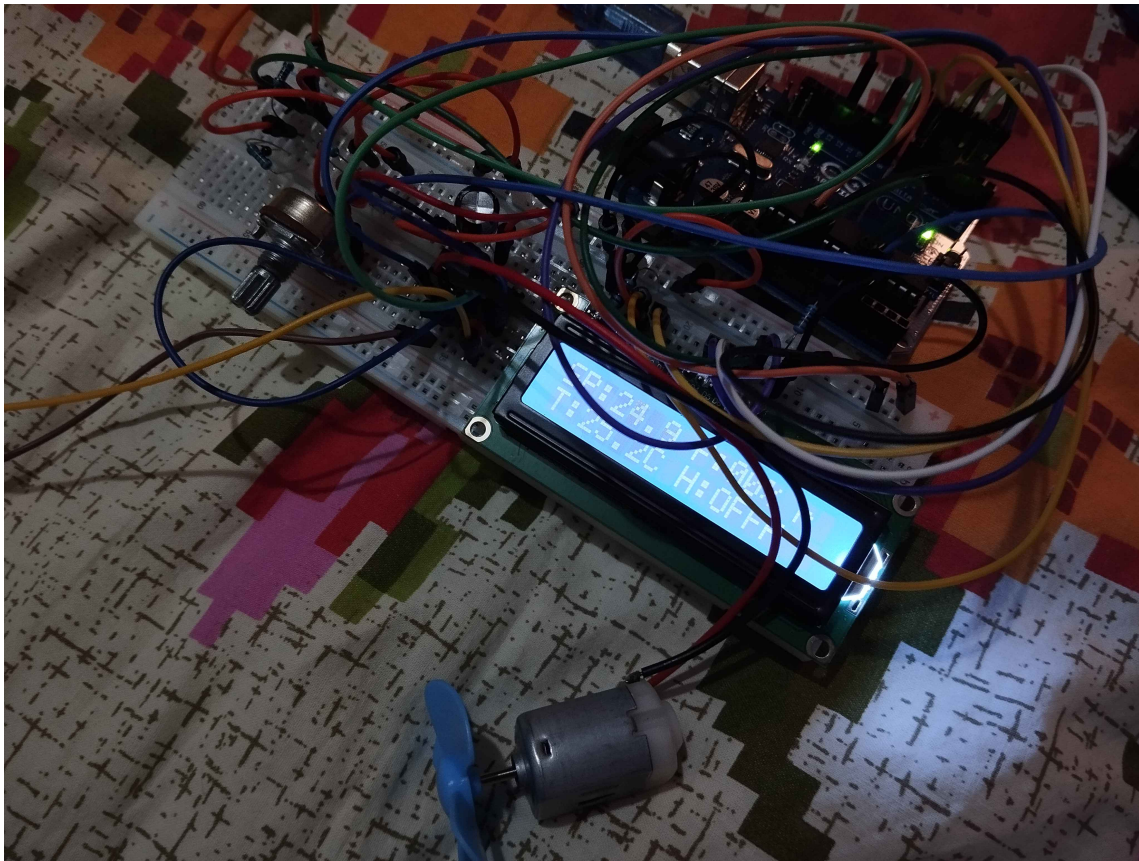


Figure 3: Hardware setup after the heater resistor is switched off

The firmware used for Part 2 is the final Lab 9 implementation after adding the L293D fan actuator and gradual fan-speed control.

3.3 Software Module Descriptions

- **ThermistorSensor**: converts ADC readings to voltage, resistance, and temperature.
- **PotentiometerInput**: maps ADC value to the configurable setpoint range.
- **OnOffHysteresisController**: implements thresholded heater logic with memory.
- **HeaterActuator**: drives the heating resistor from a digital pin.
- **L293dFanActuator**: drives and ramps the cooling fan through L293D using PWM.
- **CommandInterface**: handles serial commands such as `help` and `status`.
- **LcdDisplay**: presents local runtime status on the 1602 LCD.
- **Lab9ControlApp**: orchestrates sequential periodic tasks used by Part 2.

4 Testing and Validation

4.1 Build Results

Environment	Flash used	Flash %	RAM used	RAM %
uno	11 430 B	35.4%	788 B	38.5%

Table 4: PlatformIO build resource usage for the Part 2 firmware

4.2 Terminal Output

```
Lab9 - Thermistor heater + fan control
Commands: help | status
Plotter CSV: setpoint,temperature,heater,fan
24.5, 25.1,0,0
24.5, 25.1,0,0
24.5, 25.0,0,0
24.5, 25.0,0,0
24.5, 24.9,0,0
24.5, 24.9,0,0
24.5, 24.8,0,0
24.5, 24.8,0,0
24.5, 24.7,0,0
24.5, 24.7,0,0
24.5, 24.6,0,0
24.5, 24.6,0,0
24.5, 24.5,0,0
24.5, 24.5,0,0
24.5, 24.5,0,0
24.6, 24.4,0,0
24.5, 24.4,0,0
```

Figure 4: Serial terminal output for the Part 2 implementation

4.3 Test Scenarios and Execution Results

ID	Scenario	Observed result	Result
T-01	Startup and build	Firmware builds and starts correctly on Uno profile.	PASS
T-02	Setpoint sweep	Potentiometer changes setpoint continuously across the configured range.	PASS
T-03	Heater low temperature case	When $T \leq SP - H$, the heater turns ON.	PASS
T-04	Heater high temperature case	When $T \geq SP + H$, the heater turns OFF.	PASS
T-05	Fan start threshold	Above $SP + 1.0^\circ C$, the fan begins to spin.	PASS
T-06	Fan ramp behaviour	Fan speed increases progressively as temperature rises.	PASS
T-07	Fan slowdown behaviour	Fan speed decreases progressively as the temperature falls.	PASS
T-08	LCD/serial output	LCD and CSV output reflect current setpoint, temperature, heater, and fan state.	PASS
T-09	Resource limits	Flash and RAM remain below Uno limits.	PASS

Table 5: Test execution summary for Lab 10 Part 2

4.4 Representative Serial Output

```

35.0, 24.8,1,0
35.0, 29.2,0,0
35.0, 38.4,0,26
35.0, 42.2,0,67
35.0, 45.1,0,100

```

The four channels represent setpoint, measured temperature, heater state, and fan speed percentage.

General Conclusions

Laboratory Work #10 Part 2 extended the Part 1 thermistor-heater controller with active fan cooling. The L293D driver enabled safe motor control, while PWM ramping avoided abrupt fan-speed changes. The resulting system remains sequential, modular, observable through LCD and serial telemetry, and small enough for Arduino Uno.

Key outcomes:

1. The heater still follows ON-OFF hysteresis around the potentiometer-controlled setpoint.
2. The fan provides an additional cooling channel above the high-temperature region.
3. The fan speed changes gradually, which reduces mechanical and electrical abruptness.
4. The full system remains compatible with PlatformIO, Wokwi, and Arduino Uno limits.

Note on AI Tool Usage

AI-assisted tools were used to support coding productivity and report drafting:

- **GitHub Copilot** – code completion and refactoring assistance.
- **LLM assistance** – LaTeX drafting, editing support, and technical structure refinement.

All generated content was manually reviewed and adapted to the implemented project.

Bibliography

1. Arduino Official Reference. <https://www.arduino.cc/reference/en/>
2. AVR-libc User Manual. <https://www.nongnu.org/avr-libc/user-manual/>
3. PlatformIO Documentation. <https://docs.platformio.org/en/latest/>
4. Wokwi Simulator Documentation. <https://docs.wokwi.com/>
5. LiquidCrystal Library Reference. <https://www.arduino.cc/reference/en/libraries/liquidcrystal/>
6. L293D Motor Driver Datasheet. Texas Instruments / STMicroelectronics.

Annex – Source Code and Report Assets

The final Part 2 firmware is based on the Lab 9 implementation with the motor fan extension.

Repository URL:

<https://github.com/Ekkusuu/embedded-systems-repo/tree/main/lab9>

Lab 10 report assets are stored in:

[lab10/report/](#)